AKADEMIA NAUK STOSOWANYCH
W NOWYM SĄCZU

Wydział Nauk Inżynieryjnych
Katedra Informatyki

DOKUMENTACJA PROJEKTOWA

PROGRAMOWANIE URZĄDZEŃ MOBILNYCH

Aplikacja do trenowania

Autor:
Michał Tokarz
Paweł Pawlik

Prowadzący:
mgr inż. Dawid Kotlarski

Nowy Sącz 2026

Spis treści

1. Ogólne określenie wymagań – Wizja Produktu **3**

1.1. Filozofia Designu i Interfejs Użytkownika (UI/UX) 3
1.2. Wizualizacja postępów i Dashboard 5
1.3. Niewidzialna technologia – Funkcje w służbie biegacza 5

2. Określenie wymagań szczegółowych – Architektura i Założenia Techniczne **7**

2.1. Wymagania funkcjonalne i niefunkcjonalne 7
2.2. Architektura Systemu i Wzorzec MVVM 9
2.3. Uprawnienia systemowe, zarządzanie ekranem i natywna pozycja GPS . . 9

3. Projektowanie **13**

3.1. Wybór bibliotek i projekt struktury klas 13
3.2. Szkic layoutów i przepływ nawigacji (AppShell i PushAsync) 14
3.3. Algorytmy i pseudokody 14

4. Implementacja **20**

4.1. Implementacja lokalnej bazy danych SQLite i modeli danych 20
4.2. Implementacja autoryzacji i zapisu sesji w warstwie ViewModel 21
4.3. Optymalizacja silnika GPS, śluza asynchroniczna i krokomierz 22
4.4. Ożywienie historii: Nawigacja parametryczna i odtwarzanie trasy z pinezkami 23
4.5. Analiza wydajności kompresji JSON dla śladu GPS 24

5. Testowanie **26**

6. Podręcznik użytkownika **27**

Literatura **28**

Spis rysunków **29**

Spis tabel **30**

Spis listingów **31**

1. Ogólne określenie wymagań – Wizja Produktu

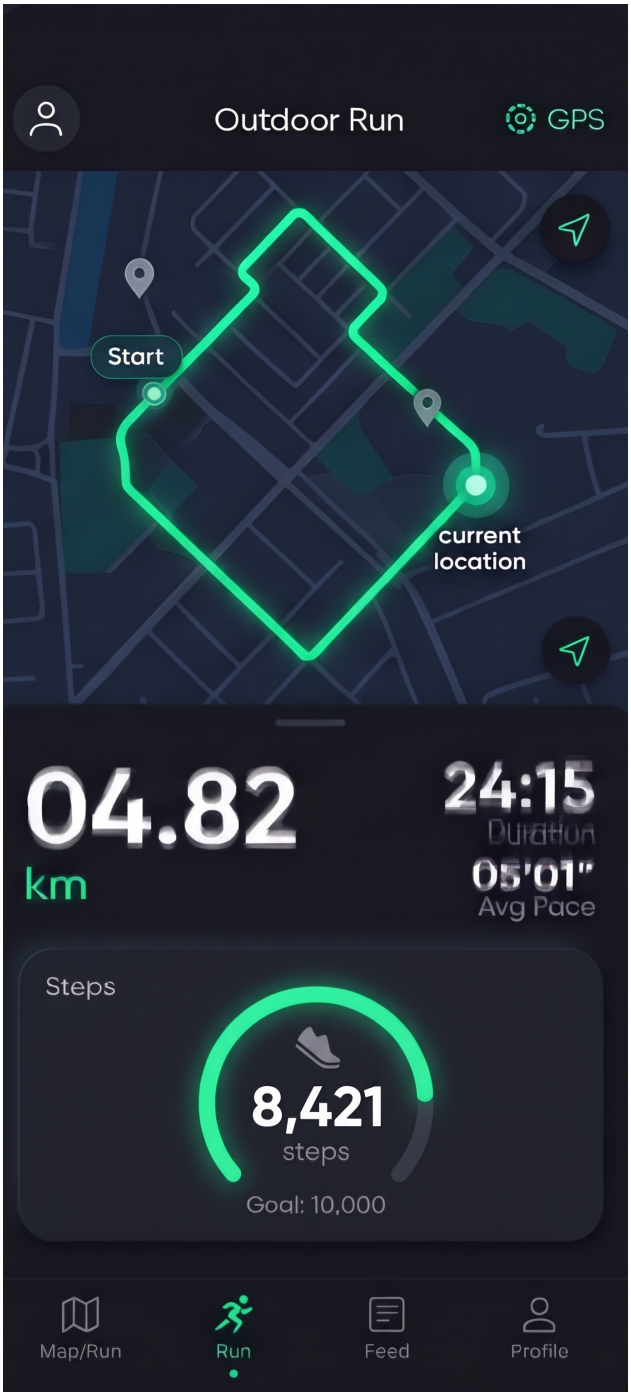
W dobie powszechnej cyfryzacji i rosnącej świadomości na temat zdrowego stylu życia, na rynku aplikacji fitness panuje ogromna konkurencja. Nasz produkt został zaprojektowany z myślą o jednym, nadrzędnym celu: dostarczeniu użytkownikowi absolutnie bezproblemowego, intuicyjnego i wizualnie dopracowanego doświadczenia, które motywuje do codziennej aktywności biegowej. Zamiast zmuszać użytkownika do dostosowywania się do technologii, sprawiliśmy, że to technologia – ukryta pod płaszczykiem nowoczesnego interfejsu – dostosowuje się do niego.

1.1. Filozofia Designu i Interfejs Użytkownika (UI/UX)

Wygląd aplikacji to pierwszy i najważniejszy punkt styku klienta z naszym produktem. Dlatego warstwa wizualna aplikacji **Runnner** została zaprojektowana w języku znaczników XAML w oparciu o najnowsze trendy projektowe ze szczególnym naciskiem na minimalizm, czytelność oraz wysoki kontrast.

Kolorystyka i typografia
Zrezygnowaliśmy z krzykliwych, rozpraszających barw na rzecz eleganckiej, głębokiej czerni (idealnie współgrającej z ekranami OLED/AMOLED, co dodatkowo oszczędza baterię smartfona) oraz energetyzujących akcentów w odcieniu neonowej, trawiastej zieleni. Ten wybór kolorystyczny nie jest przypadkowy – czerń symbolizuje profesjonalizm i skupienie, podczas gdy zieleń to uniwersalny symbol zdrowia, natury, witalności oraz pomyślnie zrealizowanego celu. Zastosowana typografia opiera się na bezszeryfowych, wyraźnych fontach, które gwarantują, że nawet w pełnym biegu, w ostrym słońcu, użytkownik bez problemu odczyta swój czas, tempo, liczbę kroków czy przebyty dystans.

Nawigacja oparta na gestach i pełny ekran mapy
Filozofia interfejsu opiera się na zasadzie: „mniej klikania, więcej biegania". W centralnym ekranie biegowym (`RunningPage`) mapa Google wypełnia 100% powierzchni roboczej ekranu, wykorzystując natywny wskaźnik pozycji użytkownika (`IsShowingUser = true`) oraz aktywną blokadę wygaszania ekranu (`KeepScreenOn = true`) podczas trwania treningu. Dolny panel statystyk został wyposażony w obsługę gestów przesunięcia (`PanGestureRecognizer`) – pociągnięcie palcem w dół płynnie chowa panel z licznikami (z wykorzystaniem animacji `CubicOut`), oddając użytkownikowi niemal cały ekran na podgląd mapy i rysowanej trasy (zob. rys. 1.1). Z kolei kliknięcie subtelnej ikony awatara w rogu mapy błyskawicznie przenosi biegacza do jego profilu osobistego za pomocą bezpiecznego routingu `AppShell`.



Rys. 1.1. Wizualizacja docelowego wyglądu interfejsu aplikacji Runnner – minimalistyczny ekran trwającego treningu z ciemnym motywem i neonowozielonym śladem trasy [↑ Spis rysunków].

1.2. Wizualizacja postępów i Dashboard

Gromadzona przez system duża ilość danych pomiarowych (zapisywana w szybkiej, lokalnej bazie danych SQLite) jest prezentowana w sposób przejrzysty i natychmiastowy:

- **Wizualny licznik kroków (Steps):** Centralne miejsce na karcie statystyk zajmuje estetyczny, półkolisty wskaźnik postępu kroków, który w czasie rzeczywistym obrazuje stopień realizacji dziennego celu biegacza.

- **Interaktywne mapy Google i rysowanie ścieżki (Polyline):** Przebyta trasa jest rysowana na żywo na natywnej mapie systemu Android (`Google Maps API`) za pomocą obiektu `Polyline` w kolorze neonowej zieleni (`#00E676`). Zoptymalizowany filtr szumów GPS o wysokiej czułości (próg 2 metrów) oraz asynchroniczna śluza zapytań sprawiają, że linia na mapie jest gładka i precyzyjnie oddaje realną ścieżkę biegacza bez obciążania procesora [1].

- **Interaktywne odtwarzanie historii tras (`SavedRoutesPage` i `RouteDetailsPage`):** Historia aktywności prezentowana jest w formie nowoczesnych kart opartych na komponencie `CollectionView`. Kliknięcie wybranego treningu na liście otwiera dedykowany ekran szczegółów (`RouteDetailsPage`), który na podstawie zapisanego w bazie ciągu JSON odtwarza przebytą trasę na mapie, oznacza pinezkami punkty **START** oraz **META** i automatycznie dopasowuje kadr kamery (`MapSpan`) tak, aby objąć cały bieg [2].

1.3. Niewidzialna technologia – Funkcje w służbie biegacza

Od strony technicznej aplikacja **Runnner** integruje geolokalizację GPS, algorytmiczny krokomierz, autoryzację kont oraz lokalną bazę danych SQLite [3]. Dla użytkownika końcowego są to jednak gotowe korzyści:

Bezpieczne konta i prywatność treningów

Aplikacja posiada wbudowany system rejestracji i logowania użytkowników na ekranie startowym. Poprawność wprowadzanego adresu e-mail jest weryfikowana zaawansowanym wyrażeniem regularnym (`Regex`), co chroni przed pomyłkami przy zakładaniu konta. Każdy zalogowany biegacz otrzymuje własną, odizolowaną przestrzeń – jego historia biegów jest powiązana z unikalnym identyfikatorem sesji (`CurrentUserId`), dzięki czemu na urządzeniu może trenować wielu domowników bez mieszania statystyk.

Precyzyjny algorytm krokomierza

Zamiast polegać wyłącznie na podatnych na błędy czujnikach wstrząsów, które potrafią naliczać kroki podczas jazdy autobusem, **Runnner** wykorzystuje zintegrowany z modułem GPS algorytm przeliczeniowy. Przyjmując średnią długość kroku biegowego na poziomie 0,75–0,80 m (1300 kroków na każdy kilometr), aplikacja gwarantuje dokładną estymację liczby kroków na każdym modelu smartfona z systemem Android, bez konieczności nadawania dodatkowych uprawnień aktywności fizycznej.

Działanie w 100% Offline (Lokalna baza SQLite)

Dzięki zastosowaniu bezserwerowej bazy danych `SQLite` (plik `Workouts.db3` w pamięci prywatnej telefonu), aplikacja nie wymaga stałego dostępu do serwerów zewnętrznych, aby zapisać trening. Biegacz może trenować w głębokim lesie lub w górach bez zasięgu sieci komórkowej, mając pewność, że po kliknięciu przycisku **STOP** jego trasa i statystyki zostaną trwale zabezpieczone w pamięci urządzenia. Zestawienie najważniejszych korzyści płynących z zastosowanych modułów prezentuje tabela 1.1.

Tab. 1.1. Przełożenie zastosowanych technologii na korzyści wizualne i UX [↑ Spis tabel].

Wykorzystany moduł	**Funkcja w aplikacji**	**Korzyść wizualna / UX dla użytkownika**
`Maui.Controls.Maps` + `IsShowingUser`	Śledzenie trasy, obiekt `Polyline` i natywna kropka GPS	Rysowana na żywo neonowozielona ścieżka na pełnoekranowej mapie bez zacięć interfejsu.
`RouteDetailsPage` + `Pin` / `MapSpan`	Wizualizacja archiwalnych biegów z formatu JSON	Interaktywna mapa ukończonego biegu z pinezkami START i META oraz automatycznym zoomem.
`PanGestureRecognizer` + `KeepScreenOn`	Zsuwany panel statystyk i blokada wygaszania ekranu	Płynne chowanie liczników gestem palca oraz brak usypiania aplikacji w kieszeni podczas biegu.
Śluza GPS (`High`) + Krokomierz	Pomiar dystansu (próg 2 m), tempa i kroków	Brak wycieków pamięci, szybki odczyt satelitów (1–2 s) i płynne naliczanie kroków.
Baza `SQLite` + `System.Text.Json`	Kompresja i zapis historii biegów	Błyskawiczny zapis całych tras GPS lokalnie na telefonie bez udziału serwera.

Podsumowując, aplikacja **Runnner** to kompletne, nowoczesne środowisko biegowe, w którym estetyka ciemnego motywu, płynne animacje interfejsu oraz prze-

myślana architektura danych tworzą produkt gotowy do codziennego, intensywnego użytkowania.

2. Określenie wymagań szczegółowych – Architektura i Założenia Techniczne

Niniejszy rozdział stanowi specyfikację techniczną projektowanego systemu **Runnner**

Aplikacja rozwijana jest w oparciu o nowoczesny ekosystem firmy Microsoft, zoptymalizowany pod kątem tworzenia natywnych aplikacji mobilnych przy użyciu jednej bazy kodu (*Single Codebase*):

- **.NET 8 MAUI (Multi-platform App UI):** Główny framework aplikacyjny zastępujący technologię Xamarin.Forms, pozwalający na kompilację kodu C# bezpośrednio do natywnych komponentów systemu Android [4].
- **Język C# oraz XAML:** C# odpowiada za pełną logikę biznesową, obsługę zdarzeń, komunikację z sensorami i bazą danych, natomiast język znaczników XAML (*eXtensible Application Markup Language*) służy do deklaratywnego budowania warstwowego interfejsu użytkownika [5].
- **SQLite (`sqlite-net-pcl`):** Lekki, transakcyjny silnik relacyjnej bazy danych działający lokalnie na urządzeniu bez udziału zewnętrznego serwera. Wszystkie tabele przechowywane są w prywatnym pliku `Workouts.db3` w katalogu `LocalApplicationData` [3].
- `System.Text.Json`**:** Wbudowany w platformę .NET wysokowydajny serializator, wykorzystywany w projekcie do kompresji złożonych list współrzędnych geograficznych (`List<Location>`) do formatu tekstowego JSON przed ich zapisem w pojedynczej kolumnie bazy SQLite [2].
- `Microsoft.Maui.Controls.Maps`**:** Oficjalna biblioteka mapowa integrująca aplikację na platformie Android bezpośrednio z natywnym pakietem Google Maps API [1].

2.1. Wymagania funkcjonalne i niefunkcjonalne

Wymagania systemowe zostały podzielone na kluczowe obszary odpowiadające modułom logicznym aplikacji. Ich zestawienie prezentuje tabela 2.1.

Tab. 2.1. Zestawienie wymagań funkcjonalnych i niefunkcjonalnych aplikacji Runnner [↑ Spis tabel].

ID	Obszar	Opis wymagania technicznego
WF-01	Rejestracja i Logowanie	Tworzenie konta w bazie SQLite z walidacją adresu e-mail poprzez wyrażenia regularne (`Regex`) oraz logowanie z przypisaniem `CurrentUserId`.
WF-02	Izolacja sesji	Filtrowanie zapisanych tras w bazie danych wyłącznie dla aktualnie zalogowanego użytkownika (`UserId`) oraz czyszczenie kolekcji po wylogowaniu.
WF-03	Śledzenie GPS i Mapa	Cykliczny odczyt lokalizacji z priorytetem `GeolocationAccuracy.High` (timeout 2 s), natywna kropka pozycji (`IsShowingUser`) i rysowanie linii `Polyline`.
WF-04	Filtr szumów i Krokomierz	Rejestracja przemieszczenia powyżej czułego progu 2 m (0,002 km) oraz przeliczanie dystansu na kroki według stałej 1300 kroków/km.
WF-05	Archiwizacja i Odtwarzanie Tras	Serializacja trasy do formatu JSON w bazie SQLite oraz jej rekonstrukcja na ekranie `RouteDetailsPage` z pinezkami `START`/`META` i centrowaniem `MapSpan`.
WN-01	Optymalizacja i Śluza GPS	Zastosowanie flagi blokującej `isChecking-Location` zapobiegającej nakładaniu się asynchronicznych zapytań GPS oraz blokady usypiania ekranu (`KeepScreenOn`).
WN-02	Wdrażanie APK	Kompilacja w trybie wydawniczym (`Release`) ze wszystkimi bibliotekami wbudowanymi bezpośrednio w podpisany pakiet instalacyjny APK.

2.2. Architektura Systemu i Wzorzec MVVM

Projekt został zorganizowany zgodnie z wzorcem architektonicznym **MVVM (Model-View-ViewModel)**, co zapewnia czystą separację warstwy prezentacji od warstwy danych i logiki biznesowej (zob. przepływ danych w tabeli 2.2):

1. **Modele (`Models`):** Klasy odwzorowujące tabele w bazie danych SQLite za pomocą atrybutów *Data Annotations* (takich jak `[PrimaryKey]`, `[AutoIncrement]`, `[Unique]` oraz `[Ignore]`). Należą do nich klasy `User`, `Workout` oraz pomocnicza klasa `RoutePoint`.

2. **Warstwa Danych (`Data`):** Klasa serwisowa `WorkoutDatabase`, inicjalizowana jako obiekt typu Singleton przy starcie aplikacji w pliku `App.xaml.cs`. Odpowiada za automatyczne tworzenie tabel oraz wykonywanie operacji CRUD.

3. **Widok-Model (`ViewModels`):** Klasa `MainViewModel` implementująca interfejs `INotifyPropertyChanged`. Przechowuje globalną instancję sesji (`CurrentUserId`, `CurrentUserName`) oraz obserwowalną kolekcję `ObservableCollection<Workout>`, która automatycznie odświeża listę tras na ekranie po każdej modyfikacji.

4. **Widoki (`Views`):** Ekrany aplikacji dziedziczące po klasie `ContentPage` (`MainPage`, `RunningPage`, `SavedRoutesPage`, `RouteDetailsPage`), spięte za pomocą dolnego paska nawigacyjnego `AppShell` oraz stosu nawigacji `Navigation.PushAsync`.

Dodatkowo w tabeli 2.3 przedstawiono szczegółowy schemat relacyjnej bazy danych `Workouts.db3`, uwzględniający typy pól, atrybuty mapowania obiektowo-relacyjnego (ORM) oraz relację jeden-do-wielu ($1 : N$) pomiędzy kontem użytkownika a jego historią biegów.

2.3. Uprawnienia systemowe, zarządzanie ekranem i natywna pozycja GPS

Do prawidłowego działania modułu mapowego oraz geolokalizacji na systemie Android, w pliku manifestu (`Platforms/Android/AndroidManifest.xml`) zadeklarowano klucz dostępowy **Google Maps API Key** oraz uprawnienia lokalizacyjne. W metodzie cyklu życia ekranu `OnAppearing()` zaimplementowano dynamiczne odpytywanie o zgodę na dostęp do GPS (`Permissions.LocationWhenInUse`).

Tab. 2.2. Schemat przepływu danych podczas sesji treningowej w aplikacji Runnner [↑ Spis tabel].

Warstwa / Moduł	**Wykonywana operacja**	**Rezultat techniczny**
1. Sensor GPS + Śluza (`Geolocation`)	Pobieranie współrzędnych w trybie `High` (timeout 2 s) pod kontrolą flagi `isCheckingLocation`.	Obiekt `Location` bez ryzyka dublowania zapytań i zamrażania wątku UI.
2. Logika Biegu (`RunningPage`)	Weryfikacja dystansu $>$ 0,002 km (2 m), obliczenie kroków ($\times$1300) i przesunięcie kamery mapy.	Płynna aktualizacja liczników UI oraz dodanie punktu do `Polyline` na mapie Google.
3. Serializacja (`System.Text.Json`)	Konwersja listy `List<Location>` przez klasę `RoutePoint`.	Skompresowany ciąg tekstowy zapisany we właściwości `RouteJson`.
4. Baza Danych (`WorkoutDatabase`)	Wywołanie asynchronicznej metody `SaveWorkoutAsync()`.	Trwały zapis rekordu powiązanego z `CurrentUserId` w pliku `Workouts.db3`.
5. Rekonstrukcja Trasy (`RouteDetailsPage`)	Deserializacja `RouteJson`, wyrysowanie `Polyline` i dodanie obiektów `Pin` (`START`/`META`).	Wizualizacja historycznej trasy wycentrowana w kadrze za pomocą `MapSpan`.

Tab. 2.3. Fizyczny schemat tabel w lokalnej bazie danych SQLite (`Workouts.db3`) [↑ Spis tabel].

Tabela	Nazwa kolumny	Typ C# / SQL	Atrybuty ORM i opis przeznaczenia
`User`	`Id`	`int` / INTEGER	`[PrimaryKey, AutoIncrement]` – główny klucz identyfikujący konto.
`User`	`FirstName`	`string` / TEXT	Imię biegacza wyświetlane w panelu powitalnym.
`User`	`LastName`	`string` / TEXT	Nazwisko użytkownika.
`User`	`Email`	`string` / TEXT	`[Unique]` – unikalny adres e-mail służący jako login.
`User`	`PasswordHash`	`string` / TEXT	Hasło dostępowe konta weryfikowane przy logowaniu.
`Workout`	`Id`	`int` / INTEGER	`[PrimaryKey, AutoIncrement]` – unikalny identyfikator treningu.
`Workout`	`UserId`	`int` / INTEGER	Klucz obcy wskazujący na pole `User.Id` właściciela biegu.
`Workout`	`Title`	`string` / TEXT	Nazwa aktywności (domyślnie „Bieg (Outdoor)”).
`Workout`	`Date`	`DateTime`	Data oraz dokładna godzina zakończenia treningu.
`Workout`	`DistanceKm`	`double` / REAL	Całkowity przebyty dystans wyrażony w kilometrach.
`Workout`	`Duration`	`TimeSpan`	Całkowity czas trwania sesji zmierzony przez stoper.
`Workout`	`PaceString`	`string` / TEXT	Sformatowane średnie tempo biegu (np. `05’12"/km`).
`Workout`	`RouteJson`	`string` / TEXT	Skompresowana do formatu JSON tablica współrzędnych GPS.
`Workout`	`Route`	`List<Location>`	`[Ignore]` – właściwość pomocnicza w pamięci RAM (pomijana przez SQLite).

Ponadto w celu optymalizacji działania aplikacji w warunkach rzeczywistego treningu wprowadzono dwa istotne rozwiązania sprzętowo-systemowe:

- **Natywna warstwa lokalizacji użytkownika:** Zrezygnowano z ręcznego renderowania sztucznych znaczników pozycji biegacza w trakcie biegu na rzecz wbudowanej właściwości kontrolki mapy `RunMap.IsShowingUser = true`. Dzięki temu system Android samodzielnie nakłada natywną, obrotową niebieską kropkę ze wskaźnikiem kierunku, co odciąża wątek renderujący MAUI.
- **Ochrona przed usypianiem procesu (`KeepScreenOn`):** Aby zapobiec usypianiu aplikacji i przerywaniu odczytów GPS po schowaniu telefonu do kieszeni, w momencie rozpoczęcia biegu aktywowana jest systemowa blokada wygaszania ekranu za pomocą właściwości `Microsoft.Maui.Devices.DeviceDisplay.Current.KeepScreenOn = true`.

3. Projektowanie

Niniejszy rozdział poświęcony jest fazie projektowania aplikacji **Runnner**.

Do realizacji projektu wykorzystano środowisko programistyczne **Visual Studio 2022** oraz pomocniczo **Android Studio** (do konfiguracji pakietów SDK i zarządzania wirtualnymi emulatorami Android Virtual Device). Wymagane było zainstalowanie pakietu SDK dla platformy **.NET 8.0** oraz obciążenia roboczego (workload) **.NET MAUI**, a także pobranie z menedżera pakietów NuGet bibliotek `sqlite-net-pcl` oraz `Microsoft.Maui.Controls.Maps`.

Kontrolę wersji zrealizowano przy pomocy rozproszonego systemu **Git**. Inicjalizacja repozytorium odbyła się lokalnie, po czym utworzono dedykowaną gałąź roboczą. Zmiany były systematycznie zatwierdzane (commitowane), a w celu uniknięcia konfliktów i zbędnego obciążenia repozytorium skonfigurowano plik `.gitignore` odrzucający pliki tymczasowe i binarne (katalogi `bin/` oraz `obj/`). Kod źródłowy był regularnie synchronizowany ze zdalnym repozytorium na platformie GitHub za pomocą polecenia `git push`.

3.1. Wybór bibliotek i projekt struktury klas

Struktura plików projektu została podzielona na cztery główne przestrzenie nazw: `Models`, `Data`, `ViewModels` oraz `Views`. Wszystkie ekrany dziedziczą po klasie bazowej `ContentPage`, natomiast główna klasa startowa `App` dziedziczy po klasie `Application`. Szczegółowy projekt klas przedstawiono w tabeli 3.1.

3.2. Szkic layoutów i przepływ nawigacji (AppShell i PushAsync)

Aplikacja wykorzystuje komponent `AppShell.xaml`, który definiuje dolny pasek nawigacyjny (*Tab Bar*) oraz bezpieczny system routingu oparty na jawnych ścieżkach tekstowych (np. `Route="ProfileTab"`), uzupełniony o klasyczną nawigację stosową (`Navigation.PushAsync`) dla ekranów podrzędnych:

1. **Ekran Profilu i Autoryzacji (`MainPage`):** Punkt wejścia dla niezalogowanego użytkownika. Umożliwia przełączanie pomiędzy widokiem logowania a formularzem rejestracji nowego konta. Po pomyślnym uwierzytelnieniu w bazie SQLite widok przełącza się na panel powitalny zalogowanego biegacza z przyciskiem wylogowania.

2. **Ekran Biegu i Mapy (`RunningPage`):** Główny ekran treningowy. Całe tło wypełnia komponent mapy Google z aktywną natywną kropką pozycji (`IsShowingUser = true`), a w lewym górnym rogu znajduje się klikalna ikona profilu wywołująca przekierowanie `Shell.Current.GoToAsync("//ProfileTab")`. W dolnej części umieszczono wysuwany panel ze wskaźnikami dystansu (km), czasu trwania (`Duration`), średniego tempa (`Avg Pace`), kołowym licznikiem kroków oraz przyciskiem **START / STOP**.

3. **Ekran Zapisanych Tras (`SavedRoutesPage`):** Zakładka prezentująca przewijaną listę kafelków ze szczegółami ukończonych biegów przypisanych wyłącznie do aktualnie zalogowanego identyfikatora `CurrentUserId`.

4. **Ekran Odtwarzania Trasy (`RouteDetailsPage`):** Widok otwierany po dotknięciu dowolnego treningu na liście historii (`Navigation.PushAsync(new RouteDetailsPage(selectedWorkout))`). Wyświetla statystyki biegu oraz interaktywną mapę z wyrysowaną trasą, markerami początku i końca biegu oraz automatycznie wyliczonym promieniem widoku.

3.3. Algorytmy i pseudokody

Poniżej przedstawiono specyfikację kluczowych algorytmów zaimplementowanych w aplikacji **Runnner**: zoptymalizowaną logikę pętli pomiarowej GPS ze śluzą asynchroniczną oraz algorytm rekonstrukcji trasy i kalkulacji kadru mapy na ekranie szczegółów treningu.

Tab. 3.1. Specyfikacja klas zaprojektowanych w architekturze aplikacji Runnner [↑ Spis tabel].

Nazwa pliku / klasy	**Warstwa**	**Przeznaczenie i główne składowe klasy**
`User.cs`	`Models`	Reprezentacja konta użytkownika w bazie SQLite. Posiada właściwości: `Id` (`[PrimaryKey, AutoIncrement]`), `FirstName`, `LastName`, `Email` (`[Unique]`) oraz `PasswordHash`.
`Workout.cs`	`Models`	Model pojedynczego treningu biegowego. Zawiera pola: `Id`, `UserId`, `Title`, `Date`, `DistanceKm`, `Duration`, `PaceString`, tekstowe pole `RouteJson` oraz ignorowaną przez bazę właściwość `Route` (`List<Location>`).
`RoutePoint.cs`	`Models`	Pomocnicza klasa przechowująca szerokość i długość geograficzną jako proste wartości typu `double` na potrzeby serializacji JSON.
`Workout Database.cs`	`Data`	Serwis utrzymujący połączenie `SQLiteAsync Connection`. Udostępnia metody: `GetWorkouts Async`, `SaveWorkoutAsync`, `GetUserAsync`, `Get-UserByEmailAsync` oraz `SaveUserAsync`.
`MainView Model.cs`	`View-Models`	Klasa wiążąca dane (Singleton `Instance`). Zawiera kolekcję `ObservableCollection< Workout>`, zmienne sesji (`CurrentUserId`, `CurrentUserName`) oraz metodę `LoadWorkouts Async()`.
`MainPage. xaml.cs`	`Views`	Zarządca formularzy logowania i rejestracji z walidacją `Regex` oraz obsługą wylogowania (`OnLogout Clicked`).
`RunningPage. xaml.cs`	`Views`	Silnik GPS, mapy i stopera. Zarządza pętlą `IDispatcherTimer`, śluzą `isCheckingLocation`, rysowaniem `Polyline`, blokadą `KeepScreenOn` oraz animacją panelu (`OnPanUpdated`).
`SavedRoutes Page.xaml.cs`	`Views`	Widok historii biegów oparty na kontrolce `CollectionView`. Obsługuje zdarzenie wyboru treningu (`OnWorkoutSelected`) i przekazuje obiekt `Workout` do widoku szczegółów.

Pseudokod 1: Logika ekranu biegania (śluzowana pętla GPS, stoper i krokomierz)

```
Zmienna isCheckingLocation = Falsz
Zmienna totalDistance = 0.0

Funkcja UpdateLocation():
    // SLUZA - ochrona przed spowolnieniami i zablokowaniem watku
    Jezeli isCheckingLocation == Prawda to: Przerwij
    isCheckingLocation = Prawda

    // Obnizenie na High zmniejsza obciazenie procesora
    location = PobierzLokalizacjeGPS(Dokladnosc: High, Max 2s)
    Jezeli location != Null oraz lastLocation != Null to:
        distance = ObliczDystansKm(lastLocation, location)

        // Czuly Filtr szumow GPS (minimum 2 metry):
        Jezeli distance > 0.002 to:
            DodajPunktDoZielonejLinii(runRoute, location)
            totalDistance = totalDistance + distance
            WyswietlNaEkranie(ZliczKroki: totalDistance * 1300)

    lastLocation = location
    isCheckingLocation = Falsz
```

Pseudokod 2: Rekonstrukcja, centrowanie i wyliczanie zasięgu na nowej mapie (RouteDetailsPage)

```
OdbierzWybranyObiektTreningu(Workout wybranyTrening)

Funkcja OdtwarzajMapeTreningu():
    Jezeli wybranyTrening.Route ma elementy to:
        Linia = Nowa Zielona Linia o grubosci 8
        Dla kazdego (lokacja) w wybranyTrening.Route:
            Linia.DodajPunkt(lokacja)

        NarysujNaMapie(Linia)

        // Zlokalizowanie kluczowych znacznikow dla wirtualnego promienia
        StartLoc = wybranyTrening.Route.PierwszyElement()
        MetaLoc = wybranyTrening.Route.OstatniElement()
        DodajPinezki("START", StartLoc) oraz ("META", MetaLoc)

        // Geometria srodka ciezkosci pomiedzy START i META
```

```
17          SrodekLat = (StartLoc.SzerokoscGeog + MetaLoc.SzerokoscGeog
    ) / 2
18          SrodekLon = (StartLoc.DlugoscGeog + MetaLoc.DlugoscGeog) /
    2
19          PromienKM = ObliczDystans(StartLoc, MetaLoc) / 2 + 0.2
20
21          SrodkujKamereMapyNaRegion(SrodekLat, SrodekLon, Proporcja:
    PromienKM)
```

W celu dokładniejszego zobrazowania działania ulepszonych algorytmów, w tabeli 3.2 rozpisano poszczególne etapy decyzyjne algorytmu filtracji sygnału GPS i estymacji kroków, natomiast w tabeli 3.3 zestawiono złożoność obliczeniową wszystkich głównych procedur w aplikacji.

Tab. 3.2. Szczegółowa specyfikacja kroków algorytmu pomiarowego GPS i krokomierza [↑ Spis tabel].

Krok	**Etap algorytmu**	**Warunek logiczny / Wzór**	**Akcja systemu**
1	Śluza asynchroniczna	`isCheckingLocation == true`	Natychmiastowe pominięcie cyklu, jeśli poprzednie zapytanie GPS jeszcze trwa.
2	Akwizycja próbki GPS	`GeolocationAccuracy.High` ($t \leq 2$ s)	Szybkie pobranie współrzędnych (ϕ, λ) bez nadmiernego drenażu baterii.
3	Obliczenie przyrostu drogi	$\Delta d = \mathrm{Dist}(P_{i-1}, P_i)$	Wyznaczenie odległości w kilometrach między kolejnymi pomiarami.
4	Czuły filtr ruchu	$\Delta d > 0{,}002$ km (2 m)	Odrzucenie szumu na postoju ($\leq$ 2 m) lub płynna rejestracja ruchu ($>$ 2 m).
5	Aktualizacja dystansu i mapy	$D_{\mathrm{total}} = D_{\mathrm{total}} + \Delta d$	Dopisanie punktu do `Polyline` i centrowanie mapy (`Radius: 0.3 km`).
6	Estymacja liczby kroków	$S = \lfloor D_{\mathrm{total}} \times 1300 \rfloor$	Płynne przeliczenie kilometrów na kroki i aktualizacja licznika `StepsLabel`.
7	Zwolnienie blokady	Blok `finally`	Ustawienie `isCheckingLocation = false`, otwierające śluzę na kolejny cykl.

Tab. 3.3. Analiza porównawcza i złożoność obliczeniowa algorytmów w aplikacji Runnner [↑ Spis tabel].

Nazwa algorytmu	**Miejsce w kodzie**	**Złożoność**	**Uzasadnienie techniczne**
Śluza GPS, filtracja (2 m) i krokomierz	`RunningPage.UpdateLocation()`	$\mathcal{O}(1)$	Stały czas weryfikacji flagi blokady i obliczenia odległości dwóch punktów.
Rekonstrukcja trasy i kadrowanie mapy	`RouteDetailsPage.ctor()`	$\mathcal{O}(n)$	Liniowa iteracja po n odczytanych z JSON punktach trasy w celu budowy `Polyline`.
Kompresja i zapis śladu GPS (JSON)	`JsonSerializer.Serialize()`	$\mathcal{O}(n)$	Czas liniowo zależny od liczby n zarejestrowanych punktów `Location` podczas biegu.
Sortowanie historii biegów użytkownika	`WorkoutDatabase.GetWorkouts Async()`	$\mathcal{O}(m \log m)$	Sortowanie malejące po dacie (`OrderByDescending`) dla m treningów w SQLite.
Weryfikacja adresu e-mail (automat skończony)	`MainPage.OnRegister Confirm()`	$\mathcal{O}(k)$	Przeszukanie łańcucha znaków o długości k za pomocą wyrażenia regularnego `Regex`.

4. Implementacja

W niniejszym rozdziale przedstawiono szczegóły implementacyjne kluczowych modułów ulepszonej aplikacji **Runnner**. Omówiono realizację bazy danych SQLite, implementację asynchronicznej śluzy chroniącej moduł GPS, obsługę zapisu treningów z izolacją sesji MVVM oraz nowatorski algorytm odtwarzania trasy na dedykowanym ekranie historii z dynamicznym przeliczaniem obszaru i nakładaniem pinesek systemowych.

4.1. Implementacja lokalnej bazy danych SQLite i modeli danych

Za trwałe przechowywanie danych na urządzeniu odpowiada klasa `WorkoutDatabase`, wykorzystująca połączenie `SQLiteAsyncConnection` z biblioteki `sqlite-net-pcl`. Baza przechowuje dwie tabele tworzone automatycznie przy inicjalizacji obiektu: `User` oraz `Workout`. Aby obsłużyć złożone kolekcje w relacyjnej bazie, w modelu `Workout.cs` wdrożono bezszwową, wewnętrzną serializację punktów trasy do ukrytego ciągu JSON.

```
using SQLite;
using System.Text.Json;

namespace Projekt_c4.Models
{
    public class Workout
    {
        [PrimaryKey, AutoIncrement]
        public int Id { get; set; }
        public int UserId { get; set; }
        public string Title { get; set; } = string.Empty;
        public DateTime Date { get; set; } = DateTime.Now;
        public double DistanceKm { get; set; }
        public TimeSpan Duration { get; set; }
        public string PaceString { get; set; } = string.Empty;
        public string RouteJson { get; set; } = string.Empty;

        [Ignore] // Wlasciwosc pomocnicza odrzucana przy zapisie
        public List<Location> Route
        {
            get => string.IsNullOrEmpty(RouteJson)
                ? new List<Location>()
                : JsonSerializer.Deserialize<List<Location>>(
```

```
                        RouteJson) ?? new List<Location>();
            set => RouteJson = JsonSerializer.Serialize(value);
        }
    }
}
```

Listing 1. Implementacja modelu `Workout.cs` z obustronną kompresją formatu JSON [↑ Spis listingów]

4.2. Implementacja autoryzacji i zapisu sesji w warstwie ViewModel

Mechanizm rejestracji użytkownika oparto na solidnej weryfikacji danych (w tym wyrażeniach regularnych e-mail). Pomyślne uwierzytelnienie odnotowuje stały identyfikator w pamięci RAM poprzez zmienną singletona, gwarantując od tej chwili poprawne wiązanie relacyjne wszystkich wykonanych sesji sportowych pod kątem bezpieczeństwa danych prywatnych.

```
// Walidacja adresu e-mail za pomoca wyrazenia regularnego
string emailPattern = @"^[^@\s]+@[^@\s]+\.[^@\s]+$";
string inputEmail = RegisterEmailEntry.Text ?? string.Empty;
if (!Regex.IsMatch(inputEmail, emailPattern))
{
    await DisplayAlert(
        "Blad", "Podaj poprawny format adresu e-mail.", "OK");
    return;
}

// Zapis nowo ukonczonego treningu dla odizolowanej sesji
var newWorkout = new Models.Workout
{
    UserId = ViewModels.MainViewModel.Instance.CurrentUserId,
    Title = "Bieg (Outdoor)",
    DistanceKm = totalDistance,
    Duration = DateTime.Now - startTime,
    RouteJson = skompresowanaTrasa,
    Date = DateTime.Now
};
await App.Database.SaveWorkoutAsync(newWorkout);
```

Listing 2. Walidacja Regex adresu e-mail i zapis treningu przypisanego do konta w bazie [↑ Spis listingów]

4.3. Optymalizacja silnika GPS, śluza asynchroniczna i krokomierz

Aby wyeliminować problem zapętlania się niewykonanych instrukcji lokalizacyjnych w module `RunningPage.xaml.cs`, na żądanie odbiornika GPS nałożono obniżenie profilu zasilania z `Best` na `High` oraz skrócony interwał `TimeSpan`. Dodatkowo wywołanie cyklu zablokowano rygorystycznym zabezpieczeniem (śluzą) w postaci flagi logicznej zapobiegającej nakładaniu się zapytań systemowych przy braku pokrycia satelitarnego. Równocześnie wdrożono wymuszenie podświetlenia zablokowanego ekranu.

```
// Flaga - sluza antyzatorowa (ochrona przed wieszaniem sie petli)
private bool isCheckingLocation = false;

private async void UpdateLocation()
{
    if (isCheckingLocation) return;
    isCheckingLocation = true;

    try
    {
        // Optymalizacja pingow: Tryb High zamiast Best
        var request = new GeolocationRequest(
            GeolocationAccuracy.High, TimeSpan.FromSeconds(2));
        var location = await Geolocation.GetLocationAsync(request);

        if (location != null && lastLocation != null)
        {
            var distance = Location.CalculateDistance(
                lastLocation, location, DistanceUnits.Kilometers);

            // Poprawiony filtr szumow - ruch > 2 metrow
            if (distance > 0.002)
            {
                runRoute.Geopath.Add(location);
                totalDistance += distance;

                // Algorytm: sztywny przelicznik krokow
                StepsLabel.Text = ((int)(totalDistance * 1300)).
   ToString("N0");
                lastLocation = location;
            }
        }
    }
```

```
33      finally
34      {
35          isCheckingLocation = false; // Rozluznienie blokady
36      }
37  }
```

Listing 3. Konstrukcja śluzy wątkowej i udoskonalony algorytm GPS o czułości 2 metrów [↑ Spis listingów]

4.4. Ożywienie historii: Nawigacja parametryczna i odtwarzanie trasy z pinezkami

Ostatnim etapem procesu produkcyjnego była implementacja przejścia użytkownika z listy jego archiwalnych biegów bezpośrednio na nowo zaprojektowany ekran historii `RouteDetailsPage`. Przekazanie wybranego z listy kafelka zostało w pełni zintegrowane ze stosem widoków. Konstruktor w oknie z mapą odkodowuje listę punktów trasy, nadaje zielony kontur ścieżce, osadza znacznik `START` na jej początku oraz marker `META` na końcu, a zarazem oblicza rozpiętość potrzebną dla silnika Google Maps, aby objąć cały trening kamerą.

```
1  private async void OnWorkoutSelected(object sender,
       SelectionChangedEventArgs e)
2  {
3      // Konwersja kliknietego obiektu listy na wlasciwy typ klasy
       Workout
4      if (e.CurrentSelection.FirstOrDefault() is Workout
       selectedWorkout)
5      {
6          ((CollectionView)sender).SelectedItem = null;
7
8          // Otwarcie nowego ekranu z przekazaniem pelnych danych o
       tym biegu
9          await Navigation.PushAsync(new RouteDetailsPage(
       selectedWorkout));
10     }
11 }
```

Listing 4. Obsługa dotknięcia karty treningu i parametryczna nawigacja stosowa [↑ Spis listingów]

```
1  public RouteDetailsPage(Workout workout)
2  {
3      InitializeComponent();
4
```

```
    // Upewnienie sie, ze tablica zawiera jakies punkty
    if (workout.Route != null && workout.Route.Any())
    {
        // 1. Odtworzenie geometrii sladu (zielona linia Polyline)
        var polyline = new Polyline { StrokeColor =
            Color.FromArgb("#00E676"), StrokeWidth = 8 };
        foreach (var loc in workout.Route) { polyline.Geopath.Add(
    loc); }
        RouteMap.MapElements.Add(polyline);

        // 2. Klasyfikacja ekstremalnych markerow kontrolnych
        var startLocation = workout.Route.First();
        var endLocation = workout.Route.Last();
        RouteMap.Pins.Add(new Pin {
            Label = "START", Location = startLocation, Type =
    PinType.Place });
        RouteMap.Pins.Add(new Pin {
            Label = "META", Location = endLocation, Type = PinType.
    Place });

        // 3. Zintegrowane przesuniecie srodka ciezkosci pola
    widzenia (Auto-Zoom)
        var centerLat = (startLocation.Latitude + endLocation.
    Latitude) / 2;
        var centerLong = (startLocation.Longitude + endLocation.
    Longitude) / 2;
        var radius = Math.Max(Location.CalculateDistance(
            startLocation, endLocation, DistanceUnits.Kilometers) /
     2 + 0.2, 0.5);

        RouteMap.MoveToRegion(MapSpan.FromCenterAndRadius(
            new Location(centerLat, centerLong), Distance.
    FromKilometers(radius)));
    }
}
```

Listing 5. Rekonstrukcja trasy w historii: dodawanie zielonej linii oraz pinezek, przy centrowaniu promienia [↑ Spis listingów]

4.5. Analiza wydajności kompresji JSON dla śladu GPS

Zastosowanie ukrytej serializacji klasy pomocniczej w obiekcie modelu pozwoliło na zadowalającą, z punktu widzenia systemów mobilnych, optymalizację rozmiaru bazy danych `Workouts.db3`. Na rysunku 4.1 przedstawiono symulacyjny wykres zależno-

ści przyrostu wagi pojedynczego pliku formatu JSON (w kilobajtach) w funkcji czasu trwania trwającej sesji biegowej.

Czas biegu (liczba próbek GPS)	**Czas zapisu**	**Rozmiar JSON**	**Wizualizacja obciążenia pamięci bazy SQLite**
15 min (∼300 pkt)	2 ms	14 KB	14 KB
30 min (∼600 pkt)	4 ms	28 KB	28 KB
45 min (∼900 pkt)	6 ms	42 KB	42 KB
60 min (∼1200 pkt)	8 ms	56 KB	56 KB

Rys. 4.1. Wykres porównawczy narzutu pamięciowego formatu JSON w bazie SQLite w zależności od czasu trwania biegu [↑ Spis tabel].

5. Testowanie

6. Podręcznik użytkownika

Bibliografia

[1] Microsoft Corporation. *Kontrolka Map w .NET MAUI (Microsoft.Maui.Controls.Maps i Google Maps API)*. 2024. URL: https://learn.microsoft.com/pl-pl/dotnet/maui/user-interface/controls/map (dostęp 25.10.2024).

[2] Microsoft Corporation. *Serializacja i deserializacja JSON w platformie .NET – System.Text.Json*. 2024. URL: https://learn.microsoft.com/pl-pl/dotnet/standard/serialization/system-text-json/overview (dostęp 25.10.2024).

[3] Frank A. Krueger. *Biblioteka sqlite-net-pcl – lokalna baza danych SQLite dla .NET*. 2024. URL: https://github.com/praeclarum/sqlite-net (dostęp 25.10.2024).

[4] Microsoft Corporation. *Dokumentacja platformy .NET Multi-platform App UI (.NET MAUI)*. 2024. URL: https://learn.microsoft.com/pl-pl/dotnet/maui/ (dostęp 25.10.2024).

[5] Microsoft Corporation. *Dokumentacja języka C oraz wzorca architektonicznego MVVM*. 2024. URL: https://learn.microsoft.com/pl-pl/dotnet/csharp/ (dostęp 25.10.2024).

Spis rysunków

1.1. Wizualizacja docelowego wyglądu interfejsu aplikacji Runnner 4

4.1. Rozmiar JSON a czas trwania biegu 25

Spis tabel

1.1. Przełożenie zastosowanych technologii na UX 6
2.1. Zestawienie wymagań funkcjonalnych i niefunkcjonalnych aplikacji Runnner 8
2.2. Schemat przepływu danych podczas sesji treningowej 10
2.3. Fizyczny schemat tabel w bazie danych SQLite 11
3.1. Specyfikacja klas zaprojektowanych w architekturze aplikacji Runnner 15
3.2. Specyfikacja kroków algorytmu GPS i krokomierza 18
3.3. Analiza złożoności algorytmów w aplikacji 19

Spis listingów

1. Implementacja `Workout.cs` 20
2. Walidacja Regex 21
3. Konstrukcja śluzy wątkowej 22
4. Obsługa dotknięcia karty 23
5. Rekonstrukcja trasy w historii 23